

HelloFresh Recipe Database

A Python tool to scrape HelloFresh recipes (from URLs or PDFs) and build a searchable database using AI-powered ingredient classification.

Features

- **Scrape HelloFresh URLs** - Automatically extract recipe data from HelloFresh links
- **Process PDF recipes** - Extract text from printed recipe PDFs
- **AI-powered classification** - Uses Claude API to intelligently extract and categorize ingredients
- **Searchable database** - SQLite database with ingredient-based search
- **Simple interface** - Easy command-line tools for setup and searching

Prerequisites

1. **Python 3.8+** (check with `python3 --version`)
2. **Claude API key** from <https://console.anthropic.com/>
 - Free tier available, or pay-as-you-go (~\$0.50-\$2 to process 100 recipes)

Quick Start

1. Install Dependencies

```
bash
pip install -r requirements.txt
```

Or install manually:

```
bash
pip install anthropic requests beautifulsoup4 PyPDF2
```

2. Set Your API Key

Mac/Linux:

```
bash
export ANTHROPIC_API_KEY='your-api-key-here'
```

Windows:

```
cmd  
  
set ANTHROPIC_API_KEY=your-api-key-here
```

Or you'll be prompted to enter it when you run the setup.

3. Add Your Recipes

Run the interactive setup:

```
bash  
  
python3 setup_recipes.py
```

This will guide you through adding recipes from:

- A text file with URLs (one per line)
- Individual URLs
- A folder of PDF files
- Individual PDF files

4. Search Your Recipes

```
bash  
  
python3 search_recipes.py
```

Then enter ingredients separated by commas:

```
Search ingredients: chicken, rice noodles, coconut milk
```

Or search directly from command line:

```
bash  
  
python3 search_recipes.py chicken "rice noodles" "coconut milk"
```

File Structure

```
recipe_processor.py # Core library (scraping, AI processing, database)  
setup_recipes.py   # Interactive setup wizard
```

```
search_recipes.py  # Recipe search interface
requirements.txt   # Python dependencies
recipes.db        # SQLite database (created automatically)
```

💡 Usage Examples

Example 1: URLs from a file

Create a text file (`my_recipes.txt`):

```
https://www.hellofresh.ca/recipes/striploin-steak-udon-noodles-68dc0672396adcc61a3e83ac
https://www.hellofresh.ca/recipes/another-recipe-123456
https://www.hellofresh.ca/recipes/yet-another-recipe-789012
```

Then run:

```
bash

python3 setup_recipes.py
# Choose option 1 and enter: my_recipes.txt
```

Example 2: Process PDFs

Place your PDF recipes in a folder (e.g., `recipe_pdfs/`), then:

```
bash

python3 setup_recipes.py
# Choose option 3 and enter: recipe_pdfs/
```

Example 3: Programmatic usage

```
python
```

```

from recipe_processor import RecipeProcessor

# Initialize
processor = RecipeProcessor(api_key="your-key-here")

# Add a recipe
processor.process_url("https://www.hellofresh.ca/recipes/...")

# Search
results = processor.search_recipes(["chicken", "rice noodles", "coconut milk"])
for recipe in results:
    print(recipe['title'])

```

How Search Works

The search looks for recipes that contain **ALL** the ingredients you specify:

- Searches are case-insensitive
- Partial matches work (e.g., "chicken" matches "chicken breast")
- Only returns recipes with every ingredient you listed

Example:

- Search: chicken, coconut milk
-  Matches: "Thai Chicken Curry" (has both)
-  No match: "Chicken Stir Fry" (missing coconut milk)

Cost Estimate

Using Claude API (Sonnet model):

- ~\$0.01-0.02 per recipe processed
- Processing 50 recipes ≈ \$0.50-\$1.00
- Processing 200 recipes ≈ \$2-\$4

This is a **one-time cost** for building your database. Searching is free (runs locally).

Troubleshooting

"No module named 'anthropic'"

- Run: pip install -r requirements.txt

"No API client available"

- Set your ANTHROPIC_API_KEY environment variable
- Or enter it when prompted by setup_recipes.py

"Error scraping URL"

- HelloFresh may have changed their website layout
- Try processing the recipe as a PDF instead (print to PDF from browser)

"No recipes found"

- Check spelling of ingredients
- Try searching with fewer ingredients
- Verify recipes were successfully added (check console output during setup)



Database Schema

The SQLite database has three tables:

recipes - Basic recipe info

- title, source_url, cook_time, servings, cuisine, protein_type

ingredients - Individual ingredients

- recipe_id, ingredient_name, quantity, category

instructions - Step-by-step directions

- recipe_id, step_number, instruction



Adding More Recipes Later

Just run `setup_recipes.py` again! It will add new recipes to your existing database without duplicating.



Next Steps

Once you have your database built, you could:

1. Build a web interface (Flask/Django)
2. Add meal planning features
3. Generate grocery lists from selected recipes

4. Export recipes to different formats
5. Add nutritional information tracking

Notes

- The AI extraction works best with well-formatted recipes
- PDFs with complex layouts may need manual verification
- Database file (`recipes.db`) can be backed up like any file
- You can view/edit the database with any SQLite viewer

Support

Questions? Check the code comments in `recipe_processor.py` for detailed documentation.

Happy cooking! 